컴퓨터 시스템 프로그래밍 — 9/08 강의 퀴즈

- **출제 범위** — 03-machine-control (슬라이드 16~27), 04-machine-procedures (슬라이드 전체)
- **구성** — OX 5문항 · 서술형 5문항 · 예제 10문항 · 심화 3문항 (총 23문항)
- **공통 전제** — 모든 어셈블리는 AT&T 문법, x86-64 System V ABI. 인자는 `%rdi %rsi %rdx %rcx %r8 %r9`, 반환은 `%rax`.
- **이름** — _____

Part 1. OX 문제 (5문항)

1. `call label` 은 다음 인스트럭션의 주소를 스택에 푸시한 뒤 `label` 로 점프하는 것과 동일하다. (O / X)

2. 인자가 7개 이상인 함수를 호출할 때, 7번째 인자를 스택에 쌓는 일은 callee가 자기 프롤로그에서 수행한다. (O / X)

3. `pushq %rbp` 는 `%rbp` 레지스터를 스택으로 옮기는 것이므로, 실행 직후 `%rbp` 는 비어 있는 상태가 된다. (O / X)

4. `ja` 와 `jb` 는 unsigned 비교에 쓰이며, `jb` 는 CF가 셋일 때 점프한다. (O / X)

5. 스택은 낮은 주소에서 높은 주소 방향으로 자라며, `%rsp` 는 항상 스택의 바닥(bottom)을 가리킨다. (O / X)

Part 2. 서술형 문제 (5문항)

각 문항은 5~8문장 분량으로 답하시오.

6. 함수의 지역 저장 공간이 왜 스택이어야 하는지를 **리엔트런시(reentrant)** 와 **라이프타임** 두 관점에서 설명하시오. 함수마다 전용 저장 공간을 하나씩 정적으로 배정하는 설계는 왜 실패하는가?

7. caller-saved 레지스터와 callee-saved 레지스터의 차이를 설명하시오. 각각 **누가, 언제** 저장하고 복원하는지, 그리고 이 두 가지 규약이 왜 둘 다 필요한지 함께 쓰시오.

8. `call` 과 `ret` 이 각각 어떤 단계로 동작하는지 push/pop 관점에서 순서대로 서술하시오. 리턴 어드레스가 무엇이며 언제 스택에 올라가고 언제 내려오는지 반드시 포함할 것.

9. ABI(Application Binary Interface)란 무엇인가? 무엇이 ABI에 포함되고 무엇은 포함되지 않는지 경계를 긋고, ABI가 서로 다른 두 바이너리를 링크했을 때 생기는 문제를 예로 들어 설명하시오.

10. `do-while` 루프가 `while` 루프보다 어셈블리로 번역하기 단순한 이유를 설명하고, 슬라이드에서 `while` 루프에 사용한 **"jump-to-middle"** 번역 방식이 무엇인지 함께 서술하시오.

Part 3. 예제 문제 (10문항)

11. leaf 함수의 `%rsp`

```
addA:
    pushq   %rbp
    movq    %rsp, %rbp
    movl    %edi, -20(%rbp)
    movl    %esi, -24(%rbp)
    movl    -20(%rbp), %edx
    movl    -24(%rbp), %eax
    addl    %edx, %eax
    movl    %eax, -4(%rbp)
    movl    -4(%rbp), %eax
    popq    %rbp
    ret
```

`addA` 진입 직전 `%rsp = 0x7010`이라 하자. 아래 표를 채우시오.

시점	%rsp	%rbp
`pushq %rbp` 직후		(호출자 값)
`movq %rsp,%rbp` 직후		
`movl %edi,-20(%rbp)` 직후		
`popq %rbp` 직후		(호출자 값)

b. 지역변수를 3개나 쓰는데 `subq $N, %rsp`가 없다. 이 함수가 값을 저장하는 주소는 `%rsp`를 기준으로 어느 쪽인가? 이것이 가능한 이유를 한 문장으로 쓰시오.

---

12. `%rsp`가 실제로 움직이는 경우

```
nonleafB:
    pushq   %rbp
    movq    %rsp, %rbp
    subq    $32, %rsp
    movl    %edi, -20(%rbp)
    movl    -20(%rbp), %edx
    movl    %edx, %eax
    addl    %eax, %eax
    addl    %edx, %eax
    movl    %eax, -4(%rbp)
    movl    -4(%rbp), %eax
    movl    %eax, %edi
    call    helper
    movl    -4(%rbp), %edx
    addl    %edx, %eax
    leave
    ret
```

a. 11번의 `addA`에는 없던 `subq $32, %rsp`가 여기엔 있다. 두 함수의 결정적 차이는 무엇인가?

b. 이 함수를 C로 복원하시오. (`int helper(int);` 선언은 주어진 것으로 본다)

---

13. 인자 7개 — 호출하는 쪽

```
callerC:
```

```
    pushq   %rbp
    movq    %rsp, %rbp
    subq    $8, %rsp
    pushq   $7
    movl    $6, %r9d
    movl    $5, %r8d
    movl    $4, %ecx
    movl    $3, %edx
    movl    $2, %esi
    movl    $1, %edi
    call    sum7
    addq    $16, %rsp
    leave
    ret
```

a. `call sum7` 실행 **직전** 시점에서, 함수 진입 시점 대비 `%rsp`는 총 몇 바이트 감소했는가? (`pushq %rbp` 포함)

b. 호출 후 `addq $16, %rsp`로 되돌리는 값이 16인 이유를 쓰시오.

---

14. 인자 7개 — 호출받는 쪽

```
sum7d:
    pushq   %rbp
    movq    %rsp, %rbp
    movq    %rdi, -8(%rbp)
    movq    %rsi, -16(%rbp)
    movq    %rdx, -24(%rbp)
    movq    %rcx, -32(%rbp)
    movq    %r8,  -40(%rbp)
    movq    %r9,  -48(%rbp)
    movq    -8(%rbp), %rdx
    ... (중략: 6개를 차례로 더함) ...
    movq    16(%rbp), %rax
    addq    %rdx, %rax
    popq    %rbp
    ret
```

a. 7번째 인자를 `16(%rbp)`에서 읽고 있다. `%rbp` 기준으로 `+8`과 `+16`에는 각각 무엇이 들어 있는가?

b. 인자가 7개인데도 이 함수에 `subq $N, %rsp`가 없는 이유를 쓰시오.

---

15. caller-saved 찾기

```
callerE:
    pushq   %rbp
    movq    %rsp, %rbp
    subq    $16, %rsp
    movl    %edi, -4(%rbp)
    movl    $1, %edi
    call    extE
    movl    -4(%rbp), %edx
    addl    %edx, %eax
    leave
    ret
```

a. 이 함수를 C로 복원하시오.

b. `movl %edi, -4(%rbp)` 한 줄이 반드시 필요한 이유를 caller-saved 개념으로 설명하시오.

16. callee-saved 찾기

```
sumrbx:
    pushq   %r12
    pushq   %rbp
    pushq   %rbx
    movq    %rdi, %r12
    movl    $0, %ebx
    movl    $0, %ebp
    jmp     .L11
.L12:
    movl    %ebx, %edi
    call    extR
    cltq
    addq    %rax, %rbp
    addq    $1, %rbx
.L11:
    cmpq    %r12, %rbx
    jl      .L12
    movq    %rbp, %rax
    popq    %rbx
    popq    %rbp
    popq    %r12
    ret
```

a. 이 함수가 저장·복원하는 callee-saved 레지스터 3개를 쓰고, 각각 무슨 용도로 쓰였는지 밝히시오.

b. `pushq` 3개로 `%rsp`는 몇 바이트 감소하는가? `popq` 순서가 `push`와 반대인 이유는?

c. 루프 변수를 `%rbx`에 둔 이유를 `call extR`과 연결해 설명하시오.

---

17. 재귀와 스택 프레임

```
fact:
    pushq   %rbp
    movq    %rsp, %rbp
    subq    $16, %rsp
    movq    %rdi, -8(%rbp)
    cmpq    $1, -8(%rbp)
    jle     .L16
    movq    -8(%rbp), %rax
    subq    $1, %rax
    movq    %rax, %rdi
    call    fact
    imulq   -8(%rbp), %rax
    jmp     .L18
.L16:
    movl    $1, %eax
.L18:
    leave
    ret
```

a. 이 함수를 C로 복원하시오.

b. `fact(4)`를 호출했을 때, 가장 깊은 시점에 `fact`의 스택 프레임은 몇 개가 동시에 존재하는가?

c. 프레임 하나당 스택 소비량을 계산하시오. (`pushq %rbp` + `subq` + `call`이 푸시하는 리턴 어드레스를 모두 포함)

18. 어셈블리 → C 복원

```
pcount_do:
    movl    $0, %edx
.L2:
    movq    %rdi, %rax
    andl    $1, %eax
    addq    %rdx, %rax
    movq    %rax, %rdx
    shrq    %rdi
    jne     .L2
    ret
```

a. 이 함수를 C로 복원하시오.

b. `jne` 앞에 `cmpq` 나 `testq` 가 없다. 그런데도 동작하는 이유는?

c. 인자 타입이 `unsigned long` 이어야 하는 이유를 `shrq` 와 연결해 설명하시오. `long` 이면 무슨 일이 생기는가?

19. 최적화 읽기

```
mul_const:
    leaq    (%rdi,%rdi,2), %rdx
    movq    %rdx, %rax
    salq    $4, %rax
    subq    %rdx, %rax
    ret
```

a. 각 줄이 `%rdx`, `%rax`에 만드는 값을 `x` (= `%rdi`)에 대한 식으로 쓰고, 최종 반환값을 구하시오.

b. 컴파일러가 `imulq` 한 줄 대신 이렇게 한 이유를 쓰시오.

---

20. 조건 코드 판정

각 코드 실행 **직후**의 CF, ZF, SF, OF 값을 채우시오. (`LONG_MAX` = `0x7FFFFFFFFFFFFFFF`)

a.

```
    movabsq $0x7FFFFFFFFFFFFFFF, %rax
    addq    $1, %rax
```

CF	ZF	SF	OF

b.

```
    movq    $-1, %rax
    addq    $1, %rax
```

CF	ZF	SF	OF

c.

```
    movq    $3, %rax
    cmpq    $5, %rax
```

CF	ZF	SF	OF

d. c. 상태에서 `jl`은 점프하는가? `jb`는? 각각의 조건식을 근거로 답하시오.

Part 4. 심화 — 흔한 함정 (3문항)

21. caller-saved의 이름이 뜻하는 것

"caller-saved 레지스터는 함수 호출 시 **항상** caller가 스택에 저장한다."

이 문장은 참인가 거짓인가? 15번의 `callerE`와, 아래 슬라이드 19의 `main`을 근거로 답하시오.

```
main:
    pushq   %rbp
    movq    %rsp, %rbp
```

```
    movl    $2, %esi
    movl    $1, %edi
    call    add
    popq    %rbp
    ret
```

`main` 도 `%edi` , `%esi` (둘 다 caller-saved)를 쓰는데 저장하는 코드가 전혀 없다. 왜인가?

---

22. `leave` 의 정체

12번과 17번의 마지막에 `leave` 가 나온다. 반면 11번과 14번은 `popq %rbp` 로 끝난다.

a. `leave` 가 수행하는 두 동작을 어셈블리 두 줄로 풀어 쓰시오.

b. 두 함수 그룹 중 `leave` 가 나온 쪽의 공통점은 무엇인가? 왜 그쪽에만 `leave` 가 필요한지 설명하시오.

---

23. 정체불명의 `subq $8, %rsp`

13번 `callerC` 를 다시 보면, 지역변수가 하나도 없는데 `subq $8, %rsp` 가 있다.

```
    pushq   %rbp          # rsp -8
    movq    %rsp, %rbp
    subq    $8, %rsp      # rsp -8   ← 왜?
    pushq   $7            # rsp -8
    ...
    call    sum7
```

함수 진입 직전 `%rsp` 가 16의 배수였다고 하자. `call sum7` 시점에 `%rsp` 를 16의 배수로 맞추기 위해 이 8바이트가 필요함을 계산으로 보이시오. (진입 시 `call` 이 푸시한 리턴 어드레스 8바이트를 잊지 말 것)

정답

Part 1. OX

1. O — 슬라이드 04-17 그대로. `call label` = `pushq %rip` (다음 인스트럭션 주소) + `jmp label`.

2. X — 7번째부터의 인자는 **caller가** 스택에 쌓아서 넘겨준다. callee는 `16(%rbp)` 등으로 **읽기만** 한다. 13번(caller의 `pushq $7`)과 14번(callee의 `movq 16(%rbp)`)을 비교하면 명확하다.

3. X — 레지스터 자체가 이동하는 게 아니라 그 안에 든 **8바이트 값이 스택에 복사**될 뿐이다. 실행 후에도 `%rbp` 값은 그대로다. 바로 다음 줄 `movq %rsp, %rbp`에서 비로소 바뀐다.

4. O — 슬라이드 03-16의 표. `jb`의 조건은 `CF`, `ja`는 `~CF & ~ZF`. unsigned 비교에서 borrow(CF)가 곧 "작다"를 뜻한다.

5. X — 두 군데가 틀렸다. 스택은 **높은 주소 → 낮은 주소**로 자라고, `%rsp`는 스택의 **탑(top)**을 가리킨다. 바텀은 스택이 시작된 가장 높은 주소 쪽이다.

Part 2. 서술형

6. 함수는 리엔트런트하므로 같은 함수가 동시에 여러 번 활성화될 수 있다. 재귀가 대표적이고, `A`가 `B`를 부르고 `B`가 다시 `A`를 부르는 경우도 있다. 함수당 저장 공간을 하나만 정적으로 배정하면 두 번째 호출이 첫 번째 호출의 인자·지역변수·리턴 어드레스를 덮어써서 첫 호출이 복귀할 수 없다. 반면 각 호출의 활성 구간은 "호출된 순간부터 리턴할 때까지"로 명확히 한정되고, 호출·복귀 순서가 반드시 LIFO(callee가 caller보다 먼저 리턴)이다. 이 라이프타임 패턴이 스택 자료구조와 정확히 일치하므로, 호출 시 프레임을 쌓고 리턴 시 걷어내는 것만으로 관리가 끝난다.

7. 두 규약은 "레지스터 값을 누가 지킬 책임이 있는가"를 나눈 것이다. **callee-saved**(`%rbx`, `%rbp`, `%r12` ~ `%r15`)는 호출된 함수가 그 레지스터를 **쓰기 전에** 자기 프레임에 백업하고 **리턴 전에** 복원한다. 따라서 caller 입장에서는 호출 전후로 값이 보존된다. **caller-saved**(`%rax`, `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` ~ `%r11`)는 callee가 마음대로 덮어써도 되고, 호출 후에도 값이 필요한 caller가 **call 전에** 스스로 저장한다. 둘 다 필요한 이유는 저장 비용을 줄이기 위해서다. 전부 callee-saved면 callee가 쓰지도 않을 레지스터까지 저장하게 되고, 전부 caller-saved면 caller가 살릴 필요 없는 값까지 저장하게 된다. 두 종류를 섞어두면 컴파일러가 값의 수명에 따라 적절한 쪽을 골라 불필요한 저장을 생략할 수 있다.

8. `call label`은 두 단계다. ① 현재 인스트럭션의 **다음** 인스트럭션 주소(= 리턴 어드레스)를 스택에 푸시하고(`%rsp` 8 감소), ② `label`로 점프한다. `ret`은 정확히 그 역순이다. ① 스택 탑에 있는 리턴 어드레스를 팝하고(`%rsp` 8 증가), ② 그 주소로 점프한다. 리턴 어드레스는 "call한 명령어의 바로 다음 인스트럭션 주소"이며, `call` 시점에 올라가고 `ret` 시점에 내려온다. push/pop이 짝을 이루므로 함수 진입 시점과 복귀 시점의 `%rsp`가 같은 값이 된다.

9. ABI는 컴파일된 바이너리끼리 상호작용하기 위한 **기계 수준의 약속**이다. 인자를 어느 레지스터로 넘기는지, 7번째부터는 스택 어디에 쌓는지, 반환값은 어디에 두는지, 어떤 레지스터가 caller-saved이고 callee-saved인지, 스택 정렬은 몇 바이트인지 등이 포함된다. 반대로 소스 코드 문법, 변수 이름, 최적화 여부처럼 컴파일 후 사라지는 것은 ABI가 아니다. ABI가 다른 두 바이너리를 링크하면 링크 자체는 성공해도 실행 시 인자가 엉뚱한 레지스터에서 읽히거나 저장돼야 할 레지스터가 날아가 값이 깨진다. 같은 x86-64여도 윈도우(`%rcx`, `%rdx`부터 시작)와 리눅스(`%rdi`, `%rsi`부터 시작)의 규약이 다른 것이 대표적인 예다.

10. `do-while`은 바디를 최소 한 번 실행하므로 "바디 → 조건 검사 → 참이면 뒤로 점프" 구조가 되고, 조건 판정과 분기가 루프 **끝에 한 번만** 필요하다. 백워드 점프 하나면 끝나므로 번역이 가장 단순하다. 반면 `while`은 바디를 한 번도 실행하지 않을 수 있어 진입 전에도 조건을 봐야 한다. **jump-to-middle**은 이를 해결하는 방식으로, 루프 시작에서 무조건 점프로 조건 검사 위치(`test:`)로 먼저 뛰고, 거기서 참이면 바디(`loop:`)로 되돌아가는 구조다. 결과적으로 `while`은 무조건 점프 1개 + 조건 점프 1개가 되며, `for`는 초기화·업데이트를 재배치해 `while`로 바꾼 뒤 같은 방식으로 번역한다.

Part 3. 예제

11.

- `pushq %rbp` 직후 — `%rsp = 0x7008`
- `movq %rsp,%rbp` 직후 — `%rsp = 0x7008`, `%rbp = 0x7008`
- `movl %edi,-20(%rbp)` 직후 — `%rsp = 0x7008` (변화 없음), `%rbp = 0x7008`
- `popq %rbp` 직후 — `%rsp = 0x7010`

b. `%rsp` 보다 **낮은 주소**에 쓴다(`-4` , `-20` , `-24` 전부 음수 오프셋). 이 함수는 leaf라 `call` 이 없고, System V ABI가 `%rsp` 아래 128바이트를 **레드 존**으로 보장하므로 `subq` 없이 그 영역을 그냥 쓸 수 있다.

12.

a. `nonleafB` 는 내부에서 `call helper` 를 한다(**non-leaf**). `call` 이 리턴 어드레스를 `%rsp` 아래에 푸시하므로 레드 존이 덮어써진다. 따라서 레드 존을 쓸 수 없고 `subq` 로 진짜 프레임을 확보해야 한다.

b.

```
int nonleafB(int x) {
    int t = x * 3;
    return helper(t) + t;
}
```

`movl %edx,%eax; addl %eax,%eax; addl %edx,%eax` 가 `x + 2x = 3x` 이다.

13.

a. `pushq %rbp` (8) + `subq $8` (8) + `pushq $7` (8) = **24바이트** 감소.

b. `call` 직전까지 `%rbp` 푸시를 제외하고 `%rsp` 를 내린 양이 `subq $8` + `pushq $7` = 16바이트이기 때문. 호출이 끝난 뒤 이 16을 되돌려야 `leave` (= `movq %rbp,%rsp`)와 `popq %rbp` 가 정상 동작한다.

14.

a. `%rbp + 8` = **리턴 어드레스**(`call` 이 푸시한 것), `%rbp + 16` = **7번째 인자**. 프롤로그에서 `pushq %rbp` → `movq %rsp,%rbp` 를 했으므로 `%rbp` 가 가리키는 곳에 old `%rbp` , 그 위에 리턴 어드레스, 그 위에 caller가 쌓은 스택 인자가 순서대로 놓인다.

b. 인자 개수와 `subq` 는 무관하다. 7번째 인자는 **caller의 프레임에** 이미 놓여 있고 callee는 읽기만 한다. 게다가 이 함수는 leaf라 6개를 저장할 48바이트도 레드 존(128바이트)에 들어간다.

15.

a.

```
int callerE(int x) {
    return extE(1) + x;
}
```

b. 인자 `x` 는 `%edi` 로 들어오는데, `extE(1)` 을 호출하려면 그 `%edi` 에 `1` 을 넣어야 한다. `%rdi` 는 caller-saved라 `extE` 가 덮어써도 되고, 호출 후 `x` 가 다시 필요하므로 **caller인 `callerE` 가 직접** 스택(`-4(%rbp)`)에 백업해둬야 한다. 이것이 caller-saved 스필이다.

16.

a. `%r12` (인자 `n` 보관), `%rbp` (누적합 `s`), `%rbx` (루프 변수 `i`). 셋 다 callee-saved이므로 함수 시작에서 push, 끝에서 pop한다.

b. `pushq` 3개 → **24바이트** 감소. 스택은 LIFO이므로 마지막에 푸시한 것이 먼저 꺼내진다. 따라서 pop 순서는 push의 역순(`%rbx` → `%rbp` → `%r12`)이어야 원래 값이 제자리로 돌아간다.

c. 루프 안에서 `call extR`을 하는데, caller-saved 레지스터에 `i`를 두면 매 호출마다 저장·복원해야 한다. callee-saved인 `%rbx`에 두면 `extR`이 알아서 지켜주므로 루프 안에서 아무것도 하지 않아도 된다. 호출을 가로질러 살아남아야 하는 값은 callee-saved에 두는 것이 정석이다.

17.

a.

```
long fact(long n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}
```

b. `fact(4)` → `fact(3)` → `fact(2)` → `fact(1)`에서 종료. 가장 깊은 시점에 **4개**의 프레임이 동시에 존재한다.

c. 리턴 어드레스 8 + `pushq %rbp` 8 + `subq $16` 16 = **프레임당 32바이트**. `fact(4)`의 최대 스택 사용량은 약 128바이트다.

18.

a.

```
long pcount_do(unsigned long x) {
    long r = 0;
    do { r += x & 1; x >>= 1; } while (x);
    return r;
}
```

`x`의 2진 표현에서 1의 개수를 센다(popcount).

b. `shrq`가 산술·논리 연산이므로 실행하면서 **ZF를 자동으로 설정**한다(implicit setting). 시프트 결과가 0이면 ZF=1이 되고 `jne`가 그 ZF를 그대로 읽는다. 별도 비교 명령이 필요 없어 한 줄을 절약한 것이다.

c. `unsigned long`이라서 컴파일러가 **논리 시프트 `shrq`**를 골랐고, 빈 최상위 비트가 0으로 채워지므로 반복하면 반드시 0이 되어 루프가 끝난다. `long`이었다면 **산술 시프트 `sarq`**가 나오고, 음수 입력에서는 사인 비트 1이 계속 복제되어 `x`가 영원히 0이 되지 않아 **무한 루프**가 된다.

19.

a.

- `leaq (%rdi,%rdi,2), %rdx` → `%rdx = x + 2x = 3x`
- `movq %rdx, %rax` → `%rax = 3x`
- `salq $4, %rax` → `%rax = 3x × 16 = 48x`
- `subq %rdx, %rax` → `%rax = 48x − 3x = **45x**`

반환값은 `45 * x`.

b. `imulq`는 레이턴시가 크다(수 사이클). 곱하는 수가 **컴파일 시점에 아는 상수**이면 `leaq`(1사이클)와 시프트·뺄셈의 조합으로 더 빠르게 만들 수 있다. 여기서는 45 = 48 − 3 = (3×16) − 3으로 분해했다.

20.

a. `LONG_MAX + 1` — **CF=0, ZF=0, SF=1, OF=1** 양수+양수인데 결과가 음수(`0x8000...0`)가 되었으므로 signed 오버플로 → OF=1, SF=1. unsigned로 보면 자리올림이 없으므로 CF=0.

b. `(-1) + 1` — **CF=1, ZF=1, SF=0, OF=0** `0xFFFF...F + 1`이 최상위 비트 밖으로 자리올림을 내보내므로 CF=1. 결과가 0이라 ZF=1. signed로는 `-1 + 1 = 0`으로 정상이므로 OF=0.

c. `cmpq $5, %rax` (rax=3) — **CF=1, ZF=0, SF=1, OF=0** `3 − 5 = −2`. borrow가 발생하므로 CF=1, 결과가 음수라 SF=1, 0이 아니라 ZF=0, signed 오버플로는 없으므로 OF=0.

d. `jl`의 조건은 `SF^OF` = `1^0` = 1 → **점프한다**(3 < 5, signed로 참). `jb`의 조건은 `CF` = 1 → **점프한다**(unsigned로도 3 < 5, 참). 이 경우엔 우연히 둘 다 참이지만, 피연산자에 음수가 섞이면 둘의 결과가 갈린다.

(a), (b), (c) 모두 실제 x86-64에서 `setc` / `setz` / `sets` / `seto`로 측정해 검증한 값이다.

Part 4. 심화

21. 거짓이다.

"caller-saved"는 "caller가 항상 저장한다"가 아니라 **"지키고 싶으면 caller가 지켜야 한다"** 는 책임의 소재를 뜻한다. 저장 여부는 그 값이 호출 이후에도 필요한지에 달렸다.

- `callerE`는 `x`(`%edi`)가 `call` 이후에도 필요하므로 `movl %edi, -4(%rbp)`로 저장했다.
- 슬라이드 19의 `main`은 `%edi`, `%esi`에 인자를 넣고 `call add`를 한 뒤 **그 값들을 다시 쓰지 않는다**. 살릴 이유가 없으니 저장 코드도 생성되지 않는다.

같은 caller-saved 레지스터인데 한쪽은 저장하고 한쪽은 안 하는 이유가 이것이다. 컴파일러는 값의 수명(live range)을 보고 필요한 경우에만 스필한다.

22.

a.

```
    movq    %rbp, %rsp      # 스택 탑을 프레임 바닥으로 되돌림
    popq    %rbp            # 저장해둔 old %rbp 복원
```

b. `leave`가 나온 쪽(12번 `nonleafB`, 17번 `fact`, 13번 `callerC`)은 모두 **`subq` 로 `%rsp`를 내렸던 함수**다. `%rsp`가 `%rbp`보다 아래로 내려가 있으므로, `popq %rbp` 하나만으로는 `%rsp`가 제자리로 돌아오지 않는다. 먼저 `%rsp`를 `%rbp` 위치로 끌어올린 뒤 pop해야 한다. 반면 11번 `addA`, 14번 `sum7d`는 `subq`가 없어 `%rsp == %rbp` 상태이므로 `popq %rbp` 한 줄이면 충분하다. `leave`는 이 두 줄을 1바이트 명령 하나로 합친 것이다.

23.

System V ABI는 `call` 실행 시점에 `%rsp` 가 **16의 배수**일 것을 요구한다.

함수 진입 직전 `%rsp ≡ 0 (mod 16)` 이라 하면, `callerC` 를 부른 `call` 이 리턴 어드레스 8바이트를 푸시했으므로 진입 시점에는

- 진입 시점: `%rsp ≡ 8 (mod 16)`
- `pushq %rbp` 후: `%rsp ≡ 0`
- `pushq $7` 만 했다면: `%rsp ≡ 8` ← **정렬 위반**
- `subq $8` 후 `pushq $7` : `%rsp ≡ 0 − 8 − 8 ≡ 0` ← 정상

즉 스택 인자 하나(8바이트)가 홀수 개라서 정렬이 깨지는 것을 `subq $8` 로 미리 보정한 것이다. 지역변수와는 무관한, 순수한 **정렬용 패딩**이다. 그래서 복구할 때도 `8 + 8 = 16` 을 한 번에 되돌린다(`addq $16, %rsp`).